# Informe de laboratorio: registros de eventos

Javier Siñuka

12 de septiembre de 2026

# Índice

**1. Introducción** **4**

**2. Objetivos** **4**

2.1. Objetivo general . . . . . 4

2.2. Objetivos específicos . . . . . 4

**3. Marco teórico** **5**

3.1. Qué es un registro de eventos . . . . . 5

3.2. Campos relevantes para análisis de seguridad . . . . . 5

3.3. Registros de acceso y registros de error . . . . . 5

3.4. Flujo de análisis de registros . . . . . 6

**4. Parte 1: descripción general de los archivos de registro** **6**

4.1. Paso 1: ejemplo de registro de un servidor web Apache . . . . . 6

4.2. Registro de acceso de muestra: `logstash-tutorial.log` . . . . . 7

4.3. Paso 2: ejemplo de registro del sistema operativo . . . . . 7

**5. Parte 2: ubicación de archivos de registro en sistemas desconocidos** **8**

5.1. 1. Consultar primero la documentación . . . . . 8

5.2. 2. Verificar que nginx se encuentre en ejecución . . . . . 8

5.3. 3. Localizar la configuración bajo `/etc` . . . . . 9

5.4. 4. Explorar `/var/log` y localizar los logs de nginx . . . . . 9

5.5. 5. Rotación de registros . . . . . 9

5.6. Resumen del procedimiento de descubrimiento . . . . . 10

**6. Parte 3: monitoreo de archivos de registro en tiempo real** **10**

6.1. Paso 1: uso de `tail` . . . . . 10

6.2. Selección del número de líneas . . . . . 10

6.3. Seguimiento continuo con `tail -f` . . . . . 10

6.4. Interpretación de códigos HTTP observados . . . . . 11

**7. Herramienta adicional: `journalctl`** **11**

7.1. systemd, journald y journalctl . . . . . 11

7.2. Filtrar por arranque . . . . . 12

7.3. Filtrar por intervalo temporal . . . . . 12

7.4. Filtrar por servicio o unidad . . . . . 12
7.5. Monitoreo en tiempo real con journalctl . . . . . 13

**8. Tabla consolidada de herramientas y comandos** **13**

**9. Resolución consolidada de las preguntas** **13**

**10. Análisis complementario desde ciberseguridad** **14**
10.1. Los logs como evidencia . . . . . 14
10.2. Correlación temporal . . . . . 14
10.3. Privilegios mínimos y acceso a logs . . . . . 14
10.4. Rotación, retención y riesgo de pérdida de evidencia . . . . . 15
10.5. Monitoreo en tiempo real frente a análisis histórico . . . . . 15

**11. Observaciones sobre precisión técnica del material** **15**

**12. Resultados de aprendizaje** **15**

**13. Conclusiones** **16**

**Referencia de la práctica** **16**

**Resumen**

El presente informe resuelve integralmente la práctica de laboratorio sobre registros de eventos en Linux, tomando como base la máquina virtual *CyberOps Workstation*. Se analizan registros de Apache y nginx, archivos de sistema ubicados en `/var/log`, métodos para localizar archivos de log en sistemas desconocidos y técnicas de monitoreo en tiempo real mediante `tail` y `journalctl`. Asimismo, se responden todas las preguntas explícitas de la guía y se incorpora un análisis complementario desde la perspectiva de ciberseguridad, orientado a monitoreo, respuesta a incidentes y análisis forense.

# 1. Introducción

Los archivos de registro constituyen una de las fuentes de evidencia más importantes en la administración de sistemas y en ciberseguridad. Servicios web, procesos del sistema operativo, mecanismos de autenticación, componentes de red y aplicaciones generan eventos que permiten reconstruir qué ocurrió, cuándo ocurrió, qué proceso intervino y, en muchos casos, qué usuario o sistema remoto estuvo involucrado.

La práctica analizada se organiza en tres partes: una descripción general de los archivos de registro, la localización de logs en sistemas cuyos detalles de instalación pueden desconocerse y el monitoreo de eventos en tiempo real. El entorno utilizado es la máquina virtual *CyberOps Workstation*, basada en Arch Linux, con servicios como nginx y `systemd-journald`.

Desde el punto de vista de un analista de seguridad, el valor de un registro no reside únicamente en "guardar mensajes", sino en aportar evidencia temporal y contextual para detectar anomalías, correlacionar eventos, investigar incidentes y validar hipótesis. Por esta razón, la sincronización horaria, la integridad de los registros, su retención y la correcta interpretación del formato resultan aspectos críticos.

# 2. Objetivos

## 2.1. Objetivo general

Comprender la estructura, ubicación, consulta y monitoreo de archivos de registro en sistemas Linux, aplicando herramientas de línea de comandos y relacionando sus resultados con tareas reales de ciberseguridad.

## 2.2. Objetivos específicos

- Interpretar entradas de registros generadas por servidores web y por el sistema operativo.
- Diferenciar registros de acceso, registros de error y registros generales del sistema.
- Ubicar archivos de configuración y registros de nginx cuando no se conoce inicialmente su ruta exacta.
- Utilizar `cat`, `more`, `less`, `ps`, `grep`, `ls` y `tail` para inspeccionar información operativa.
- Comprender el funcionamiento de la rotación de registros.
- Monitorear logs en tiempo real mediante `tail -f` y `journalctl -f`.
- Aplicar filtros de `journalctl` por arranque, intervalo temporal y unidad de `systemd`.

- Resolver las preguntas planteadas en la guía con fundamento técnico y sin atribuir evidencia que no se encuentre en el material proporcionado.

# 3. Marco teórico

## 3.1. Qué es un registro de eventos

Un archivo de registro o *log* almacena eventos generados por software, servicios, procesos del sistema operativo o transacciones entre componentes. El formato concreto depende de la aplicación que produce el evento. Por ello, no existe un único formato universal: cada servicio puede definir qué campos registrar, cómo ordenarlos y en qué destino almacenarlos.

En Linux es habitual encontrar registros en `/var/log`, mientras que los archivos de configuración suelen ubicarse bajo `/etc`. Estas rutas son convenciones frecuentes, no garantías absolutas. La configuración de cada aplicación puede modificar el destino de los logs.

## 3.2. Campos relevantes para análisis de seguridad

Aunque la estructura varía entre productos, los registros útiles para ciberseguridad suelen contener una combinación de los siguientes datos:

- marca temporal;
- host o servicio que genera el evento;
- severidad o tipo de evento;
- identificador de proceso o hilo;
- identidad de usuario o cuenta de servicio;
- dirección IP de origen o destino;
- recurso solicitado;
- resultado de la operación;
- descripción del evento o mensaje de error.

La marca temporal merece especial atención. La correlación entre sistemas diferentes solo es fiable si sus relojes están razonablemente sincronizados. Una diferencia horaria puede alterar la secuencia aparente de un incidente y dificultar la reconstrucción forense.

## 3.3. Registros de acceso y registros de error

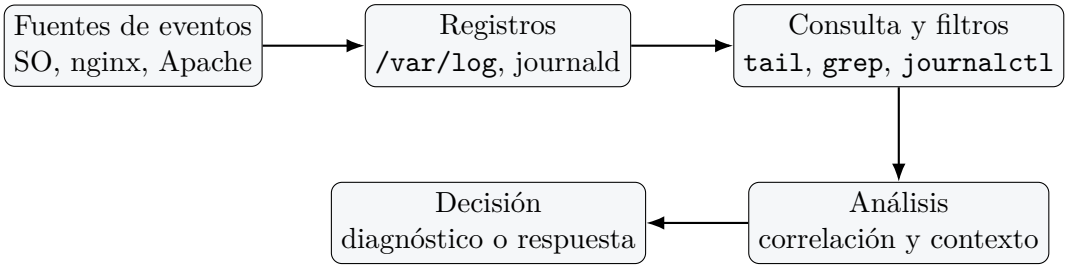
En un servidor web conviene distinguir dos categorías habituales:

- **Registro de acceso:** documenta solicitudes HTTP recibidas y normalmente incluye origen, fecha y hora, método HTTP, recurso, versión del protocolo, código de estado, tamaño de respuesta y agente de usuario.

- **Registro de error:** documenta fallos operativos o condiciones anómalas del servidor, como archivos inexistentes, errores de configuración, permisos insuficientes o problemas durante la ejecución.

La diferencia es fundamental: una misma interacción puede producir una entrada de acceso y, si ocurre un problema, otra entrada en el registro de error.

### 3.4. Flujo de análisis de registros




## 4. Parte 1: descripción general de los archivos de registro

### 4.1. Paso 1: ejemplo de registro de un servidor web Apache

La guía proporciona la siguiente entrada de error de Apache:

```
[Wed Mar 22 11:23:12.207022 2017] [core:error]
[pid 3548:tid 4682351596] [client 209.165.200.230]
File does not exist: /var/www/apache/htdocs/favicon.ico
```

Esta entrada puede descomponerse de la siguiente manera:

| Campo | Interpretación |
|---|---|
| Marca temporal | Miércoles 22 de marzo de 2017 a las 11:23:12.207022. |
| Tipo | `core:error`: evento clasificado como error del núcleo de Apache. |
| PID/TID | El proceso tiene PID 3548 y se registra además el identificador de hilo 4682351596. |
| Cliente | La solicitud se asocia con la dirección IP `209.165.200.230`. |
| Descripción | Apache intentó localizar `/var/www/apache/htdocs/favicon.ico`, pero el archivo no existía. |

#### Respuesta a la pregunta 1

#### ¿Qué sucedió?

Un cliente con dirección IP `209.165.200.230` realizó una solicitud que llevó al servidor Apache a intentar acceder al archivo `/var/www/apache/htdocs/favicon.ico`. El archivo no existía en esa ruta, por lo que Apache registró un evento de tipo `core:error`. El incidente ocurrió el 22 de marzo de 2017 a las 11:23:12 y fue procesado por el proceso Apache con PID 3548.

Desde el punto de vista de seguridad, esta única entrada no demuestra actividad maliciosa. La ausencia de `favicon.ico` es un evento frecuente en servidores web. Para evaluar riesgo sería

necesario correlacionar el origen con otras solicitudes, frecuencia, recursos consultados, códigos de respuesta y comportamiento temporal.

## 4.2. Registro de acceso de muestra: `logstash-tutorial.log`

La guía utiliza el siguiente comando:

```
cat /var/log/logstash-tutorial.log
```

Entre las entradas mostradas aparecen solicitudes como:

```
83.149.9.216 - - [04/Jan/2015:05:13:42 +0000]
"GET /presentations/logstash-monitorama-2013/images/kibana-search.png HTTP/1.1"
200 203023
"http://semicomplete.com/presentations/logstash-monitorama-2013/"
"Mozilla/5.0 (...) Chrome/32.0.1700.77 Safari/537.36"
```

Los elementos relevantes son la IP de origen, la marca temporal, el método HTTP, el recurso solicitado, la versión del protocolo, el código HTTP, el tamaño de la respuesta, el referente y el agente de usuario.

### Respuesta a la pregunta 2

### ¿La salida puede considerarse una transacción web? ¿Por qué el formato es diferente del ejemplo anterior?

Sí. Cada entrada del archivo representa una solicitud HTTP y su resultado, es decir, una interacción o transacción web registrada por el servidor. En el ejemplo se observa, por ejemplo, una petición `GET` y una respuesta con código `200`.

La diferencia de formato **no es causada por el comando `cat`**. `cat` se limita a mostrar el contenido del archivo sin reinterpretarlo. El formato cambia porque se están observando registros de naturaleza distinta: la primera entrada corresponde a un **registro de error de Apache**, mientras que `/var/log/logstash-tutorial.log` presenta un formato de **registro de acceso web**. Además, cada servidor y cada configuración puede definir su propio formato de logging.

## 4.3. Paso 2: ejemplo de registro del sistema operativo

La práctica consulta `/var/log/messages` mediante:

```
sudo more /var/log/messages
```

La salida incluye mensajes del kernel, por ejemplo eventos de la interfaz `enp0s3`:

```
Mar 20 14:28:29 secOps kernel: ... enp0s3: link down
Mar 20 14:28:33 secOps kernel: ... enp0s3: link up, 100Mbps, full-duplex
Mar 20 14:28:35 secOps kernel: ... enp0s3: link down
Mar 20 14:28:43 secOps kernel: ... enp0s3: link up, 100Mbps, full-duplex
Mar 20 14:28:53 secOps kernel: ... enp0s3: link down
Mar 20 14:28:57 secOps kernel: ... enp0s3: link up, 100Mbps, full-duplex
Mar 20 14:29:01 secOps kernel: ... enp0s3: link down
Mar 20 14:29:05 secOps kernel: ... enp0s3: link up, 100Mbps, full-duplex
```

La secuencia evidencia una inestabilidad o *flapping* del enlace: la interfaz alterna repetidamente entre estados `down` y `up`. Una situación así puede provocar pérdida de conectividad, retransmisiones, interrupciones de sesiones y percepción de lentitud.

**Respuesta a la pregunta 3**

**¿Hay pruebas en las entradas mostradas de lentitud alrededor de las 04:20 del 19 de mayo?**

**No es posible demostrarlo con el fragmento de registro mostrado en la guía.** Las líneas visibles con eventos de red corresponden al **20 de marzo alrededor de las 14:28–14:29**, y posteriormente aparecen eventos del **22 de marzo**. No se muestran entradas del 19 de mayo a las 04:20.

Sí existe evidencia de un problema de enlace en las líneas de marzo citadas anteriormente, pero atribuirlo al incidente reportado para mayo sería incorrecto por incompatibilidad temporal. En una investigación real se debería consultar el archivo completo alrededor del intervalo informado, por ejemplo:

```
sudo grep '^May 19 04:2' /var/log/messages
sudo grep -E '^May 19 04:2.*(enp0s3|link|network|dhcp)' /var/log/messages
```

La conclusión correcta es, por tanto, que el extracto visible contiene un patrón técnicamente compatible con problemas de conectividad, pero **no constituye evidencia del incidente fechado el 19 de mayo**.

# 5. Parte 2: ubicación de archivos de registro en sistemas desconocidos

## 5.1. 1. Consultar primero la documentación

Cuando se trabaja con un servicio que no se conoce, la estrategia inicial debe ser revisar su documentación. La práctica utiliza:

```
man nginx
```

La página de manual identifica a nginx como servidor HTTP, proxy inverso y proxy de correo. Además, confirma que dispone de registro de depuración y referencias a rutas de archivos como el PID, el archivo principal de configuración y el registro de errores.

Este paso es importante metodológicamente: buscar directamente por todo el sistema puede ser más lento y generar ruido. La documentación permite conocer convenciones, parámetros de compilación y archivos relevantes antes de realizar búsquedas más amplias.

## 5.2. 2. Verificar que nginx se encuentre en ejecución

La guía combina `ps` con `grep`:

```
ps ax | grep nginx
```

La salida mostrada contiene un proceso maestro y un proceso *worker*, además del propio `grep`. Se observa que nginx fue iniciado con parámetros que fijan el PID en `/run/nginx.pid` y redirigen el registro de errores a `stderr`.

**Interpretación de seguridad:** verificar procesos activos permite confirmar que el servicio investigado efectivamente está en ejecución y proporciona datos como PID, usuario, argumentos de inicio y opciones relevantes. Los argumentos de línea de comandos pueden revelar rutas de configuración personalizadas o comportamientos que no coinciden con los valores predeterminados.

## 5.3. 3. Localizar la configuración bajo `/etc`

La práctica enumera `/etc` y detecta el subdirectorio `/etc/nginx`. Posteriormente ejecuta:

```
ls -l /etc/nginx/
```

Entre los archivos se encuentra `/etc/nginx/nginx.conf`, además de configuraciones auxiliares y archivos personalizados. Luego se consulta:

```
cat /etc/nginx/nginx.conf
```

La configuración mostrada contiene líneas de `error_log` comentadas. Como las líneas precedidas por `\#` no se encuentran activas, no se obtiene allí una ruta explícita efectiva de logging para esa ejecución, por lo que resulta razonable comprobar las ubicaciones predeterminadas.

## 5.4. 4. Explorar `/var/log` y localizar los logs de nginx

La guía ejecuta:

```
ls -l /var/log/
sudo ls -l /var/log/nginx
```

En `/var/log` se observa el subdirectorio `/var/log/nginx`. En él aparecen:

```
access.log
access.log.1.gz
access.log.2.gz
access.log.3.gz
access.log.4.gz
```

El archivo `/var/log/nginx/access.log` contiene los eventos más recientes, mientras que los archivos comprimidos numerados son registros rotados.

## 5.5. 5. Rotación de registros

La rotación evita que un único archivo crezca indefinidamente. De forma general, el mecanismo conserva versiones antiguas, puede comprimirlas y crea o reutiliza un archivo principal para los eventos nuevos.

Para un analista de seguridad esto tiene varias consecuencias:

- la ausencia de eventos antiguos en `access.log` no implica que nunca hayan existido;
- es necesario revisar los archivos rotados cuando el intervalo de investigación es anterior;

- los archivos comprimidos pueden requerir herramientas como `zgrep`, `zcat` o descompresión controlada;
- las políticas de retención determinan cuánto tiempo permanece disponible la evidencia local.

### 5.6. Resumen del procedimiento de descubrimiento

1. Consultar documentación y página de manual del servicio.
2. Confirmar si el proceso se está ejecutando y con qué argumentos.
3. Localizar el archivo de configuración principal.
4. Revisar directivas activas relacionadas con logs.
5. Aplicar convenciones del sistema, como `/var/log`, cuando la configuración no especifica una ruta alternativa.
6. Verificar la actividad del archivo generando eventos controlados y observando si el log se actualiza.

Este último punto es especialmente sólido: no basta con encontrar un archivo cuyo nombre "parezca correcto"; conviene demostrar que el servicio lo actualiza cuando ocurre una operación conocida.

## 6. Parte 3: monitoreo de archivos de registro en tiempo real

### 6.1. Paso 1: uso de `tail`

Por defecto, `tail` muestra las últimas diez líneas de un archivo. La práctica utiliza:

```
sudo tail /var/log/nginx/access.log
```

Las entradas mostradas registran solicitudes locales desde `127.0.0.1`, incluyendo respuestas HTTP `200`, `304` y `404`. Entre ellas aparece el acceso a `/favicon.ico` con código `404`, lo que indica que el recurso solicitado no fue encontrado.

### 6.2. Selección del número de líneas

La opción `-n` permite indicar cuántas líneas finales deben mostrarse:

```
sudo tail -n 5 /var/log/nginx/access.log
```

Esto resulta útil para obtener contexto reciente sin cargar un archivo potencialmente grande.

### 6.3. Seguimiento continuo con `tail -f`

Para monitoreo en tiempo real:

```
sudo tail -f /var/log/nginx/access.log
```

La opción `-f` (*follow*) mantiene el proceso activo y muestra nuevas líneas a medida que se anexan al archivo. Al abrir `127.0.0.1` en el navegador se genera una nueva solicitud al servidor local y la correspondiente entrada aparece en el terminal.

La observación de esta actualización permite confirmar empíricamente que `/var/log/nginx/access.log` es utilizado por nginx como registro de acceso en ese entorno.

**Nota sobre el material:** en una línea de la guía aparece la ruta `/var/log/acess.log`; por el contexto, los comandos anteriores y posteriores, la ruta operativa utilizada en la práctica es `/var/log/nginx/access.log`. En este informe se conserva esta última y se deja explícita la discrepancia tipográfica.

### 6.4. Interpretación de códigos HTTP observados

| Código | Significado | Interpretación en la práctica |
|---|---|---|
| 200 | OK | El servidor respondió satisfactoriamente y entregó el recurso solicitado. |
| 304 | Not Modified | El recurso no cambió respecto de la copia en caché del cliente; por ello no se retransmite el contenido completo. |
| 404 | Not Found | El recurso solicitado no existe en la ubicación esperada; en los ejemplos ocurre con `/favicon.ico`. |

Estos códigos son informativos, pero en análisis de seguridad deben interpretarse con contexto. Numerosos `404` secuenciales sobre rutas sensibles o aleatorias podrían indicar enumeración de contenidos, mientras que un `404` aislado para `favicon.ico` es habitualmente benigno.

## 7. Herramienta adicional: `journalctl`

### 7.1. systemd, journald y journalctl

La máquina virtual utiliza `systemd` como sistema de inicialización. Dentro de este ecosistema, `systemd-journald` recopila eventos y los almacena en un formato propio. `journalctl` es la utilidad de consulta del *journal*.

La guía comienza con:

```
journalctl
```

El mensaje mostrado advierte que el usuario `analyst` no ve necesariamente los eventos de otros usuarios ni todos los mensajes del sistema porque no pertenece a grupos con acceso ampliado.

#### Respuesta a la pregunta 4

#### ¿Cómo ejecutar `journalctl` y ver todas las entradas de registro?

La solución inmediata en la práctica es ejecutar la herramienta con privilegios elevados:

```
sudo journalctl
```

El mensaje de `journalctl` también indica que usuarios pertenecientes a grupos como `adm`, `systemd-journal` o `wheel` pueden disponer de acceso ampliado según la configuración del

sistema. Para el laboratorio, el uso de `sudo` es la respuesta directa y coherente con los comandos posteriores.

## 7.2. Filtrar por arranque

La opción `-b` restringe la salida al arranque seleccionado:

```
sudo journalctl -b
```

Para consultar arranques anteriores pueden emplearse desplazamientos negativos. La guía muestra el segundo arranque anterior:

```
sudo journalctl -b -2
```

Y para listar los arranques disponibles:

```
sudo journalctl --list-boots
```

Cada entrada de `-list-boots` incluye un desplazamiento, un identificador de arranque y el intervalo temporal correspondiente. Esto es muy útil durante análisis post-incidente, ya que permite acotar una investigación a la sesión de arranque relevante.

## 7.3. Filtrar por intervalo temporal

La guía utiliza `-since` para limitar la consulta:

```
sudo journalctl --since "2 hours ago"
sudo journalctl --since "1 day ago"
```

En un análisis real, el filtrado temporal reduce drásticamente el volumen de información y permite correlacionar un reporte de incidente con eventos del sistema ocurridos en la misma ventana.

Como extensión natural, cuando la herramienta y versión lo permiten, puede combinarse `-since` con `-until` para definir un intervalo cerrado, por ejemplo:

```
sudo journalctl --since "2017-05-19 04:15:00" \
                --until "2017-05-19 04:25:00"
```

Este último comando se presenta como técnica complementaria de análisis; no forma parte de los comandos exigidos explícitamente por la práctica.

## 7.4. Filtrar por servicio o unidad

Para mostrar eventos asociados con nginx:

```
sudo journalctl -u nginx.service
```

La salida de la guía contiene eventos de inicio y detención del servicio, advertencias de configuración, referencias al PID y errores relacionados con `favicon.ico`. El filtro por unidad evita revisar mensajes no relacionados provenientes de todo el sistema.

### 7.5. Monitoreo en tiempo real con journalctl

El equivalente funcional al seguimiento continuo de `tail -f` es:

```
sudo journalctl -f
```

Y para restringir el seguimiento exclusivamente a nginx:

```
sudo journalctl -u nginx.service -f
```

Al mantener este comando activo y abrir `127.0.0.1` o `127.0.0.1:8080`, la guía indica que pueden observarse en tiempo real eventos relacionados con el servicio, incluido el error por el archivo `favicon.ico` inexistente.

## 8. Tabla consolidada de herramientas y comandos

| Comando | Función en la práctica |
|---|---|
| `cat archivo` | Muestra el contenido completo de un archivo de texto en la salida estándar. |
| `more archivo` | Permite recorrer un archivo progresivamente, útil cuando el contenido no cabe en pantalla. |
| `less archivo` | Visor interactivo con navegación y búsqueda; adecuado para inspección de archivos extensos. |
| `ps ax \| grep nginx` | Identifica procesos nginx activos y permite observar sus argumentos de ejecución. |
| `ls -l ruta` | Enumera archivos, permisos, propietarios, tamaños y fechas. |
| `tail archivo` | Muestra las últimas diez líneas de un archivo por defecto. |
| `tail -n 5 archivo` | Muestra las últimas cinco líneas. |
| `tail -f archivo` | Sigue el archivo y muestra nuevas líneas en tiempo real. |
| `journalctl` | Consulta los registros administrados por journald accesibles al usuario actual. |
| `sudo journalctl` | Consulta el journal con privilegios elevados. |
| `journalctl -b` | Filtra por arranque. |
| `journalctl -list-boots` | Lista los arranques registrados y sus identificadores. |
| `journalctl -since ...` | Filtra por tiempo de inicio de la consulta. |
| `journalctl -u nginx.service` | Filtra los eventos de la unidad nginx. |
| `journalctl -f` | Sigue nuevas entradas del journal en tiempo real. |
| `journalctl -u nginx.service -f` | Combina filtro por unidad con monitoreo en tiempo real. |

## 9. Resolución consolidada de las preguntas

1. **Entrada de Apache:** el cliente `209.165.200.230` provocó una solicitud que hizo que Apache buscara `/var/www/apache/htdocs/favicon.ico`; el recurso no existía y se registró un error `core:error` asociado al PID 3548.

2. **Transacción web y diferencia de formato:** las entradas de `logstash-tutorial.log` representan solicitudes HTTP y sus respuestas, por lo que corresponden a transacciones web

registradas. El formato no cambia por `cat`; cambia porque se trata de un registro de acceso, mientras que el primer ejemplo era un registro de error, y cada aplicación/configuración define su formato.

3. **Supuesto problema de red del 19 de mayo a las 04:20:** el fragmento visible no contiene eventos de ese momento, por lo que no puede confirmarse el reporte a partir de las líneas mostradas. Sí se observan repetidos `link down`/`link up` el 20 de marzo alrededor de las 14:28–14:29, lo cual evidencia inestabilidad de enlace en otra fecha.

4. **Acceso completo con journalctl:** ejecutar `sudo journalctl`. Alternativamente, el acceso puede depender de pertenecer a grupos autorizados como `adm`, `systemd-journal` o `wheel`, según la política del sistema.

# 10. Análisis complementario desde ciberseguridad

## 10.1. Los logs como evidencia

Un registro es una fuente de evidencia, pero debe interpretarse con criterios de confiabilidad. El analista debe preguntarse quién genera el evento, si la fuente puede ser manipulada, si el reloj es correcto, si existe pérdida de datos por rotación y si el registro local fue alterado después del incidente.

En investigaciones formales conviene preservar copias, registrar hashes criptográficos de los archivos recolectados y mantener trazabilidad sobre quién accedió a la evidencia. Estas prácticas no forman parte operativa del laboratorio, pero son una extensión profesional natural del trabajo con logs.

## 10.2. Correlación temporal

La pregunta sobre una supuesta lentitud de red ilustra un principio central: **un evento técnicamente plausible no es evidencia suficiente si la marca temporal no coincide**. Un analista no debería seleccionar un evento "parecido" y atribuirlo al incidente sin correspondencia temporal.

La correlación exige alinear eventos provenientes de múltiples fuentes: registros del kernel, servidor web, autenticación, firewall, DNS, aplicaciones y, cuando corresponde, telemetría de red. De allí la importancia de NTP u otro mecanismo confiable de sincronización horaria.

## 10.3. Privilegios mínimos y acceso a logs

La necesidad de utilizar `sudo` muestra que los registros pueden contener información sensible. Conceder lectura irrestricta a todos los usuarios puede exponer nombres de cuenta, rutas internas, direcciones IP, errores de aplicaciones y otros datos útiles para un atacante.

El principio de mínimo privilegio recomienda que solo las cuentas y grupos que requieran acceso operativo puedan consultar registros sensibles. Al mismo tiempo, un SOC necesita acceso suficiente para investigar incidentes; por ello es habitual separar funciones mediante grupos, roles o plataformas centralizadas de logging.

### 10.4. Rotación, retención y riesgo de pérdida de evidencia

La rotación controla el consumo de disco, pero una política demasiado agresiva puede eliminar evidencia antes de que un incidente sea detectado. Una política adecuada debe equilibrar capacidad de almacenamiento, requisitos operativos, obligaciones normativas y ventana de investigación.

En entornos de mayor criticidad, los registros suelen enviarse a un sistema remoto o SIEM. Esto reduce la dependencia del almacenamiento local y dificulta que un atacante que compromete un host elimine toda la evidencia disponible.

### 10.5. Monitoreo en tiempo real frente a análisis histórico

`tail -f` y `journalctl -f` son excelentes para observar qué ocurre mientras se reproduce un evento o se diagnostica un servicio. Sin embargo, el monitoreo interactivo manual no sustituye un sistema de detección automatizada.

Un flujo profesional combina:

- **análisis histórico**, para reconstruir hechos ya ocurridos;
- **monitoreo en tiempo real**, para observar cambios mientras suceden;
- **correlación**, para relacionar múltiples fuentes;
- **alertamiento**, para identificar automáticamente patrones de interés;
- **retención**, para conservar evidencia durante el período necesario.

## 11. Observaciones sobre precisión técnica del material

Para conservar rigurosidad académica, se normalizan algunos detalles tipográficos del documento original:

- Los comandos se escriben con guiones ASCII reales, por ejemplo `-list-boots` y `-since`; en el PDF algunos guiones aparecen tipográficamente como rayas.
- Se utiliza `/var/log/nginx/access.log` como ruta del registro de acceso de nginx, coherente con los comandos y listados previos. El texto contiene una mención aislada a `/var/log/acess.log`, aparentemente incompleta.
- La guía contiene una forma unida `sudonginx` en una nota; la sintaxis operativa correcta requiere separar `sudo` del comando nginx.

Estas correcciones de presentación no modifican el objetivo ni los resultados conceptuales de la práctica.

## 12. Resultados de aprendizaje

A partir de la resolución completa del laboratorio se obtienen los siguientes resultados:

1. Se interpretó correctamente la estructura de registros web y del sistema operativo.

2. Se diferenciaron registros de acceso y registros de error.
3. Se estableció un método reproducible para localizar logs en un sistema inicialmente desconocido.
4. Se comprendió el propósito de la rotación de registros y su efecto sobre investigaciones históricas.
5. Se utilizaron conceptualmente herramientas de inspección estática y seguimiento en tiempo real.
6. Se aplicaron filtros de `journalctl` por arranque, tiempo y unidad de servicio.
7. Se resolvieron las cuatro preguntas explícitas de la guía con fundamento en la evidencia efectivamente mostrada.
8. Se vinculó el laboratorio con principios profesionales de correlación, integridad, mínimo privilegio y preservación de evidencia.

## 13. Conclusiones

La práctica demuestra que el trabajo con registros no consiste únicamente en abrir archivos de texto, sino en comprender el origen, el formato, la ubicación y el contexto de cada evento. La combinación de herramientas sencillas como `cat`, `more`, `grep`, `tail` y `ps` con utilidades específicas como `journalctl` permite investigar el comportamiento de un sistema Linux con gran nivel de detalle.

El procedimiento utilizado para nginx es especialmente valioso: consultar documentación, verificar el proceso, identificar la configuración, explorar ubicaciones convencionales y validar el archivo generando una solicitud conocida. Esta secuencia evita asumir que una ruta es correcta únicamente por su nombre.

Desde la perspectiva de ciberseguridad, los principales aprendizajes son la necesidad de correlacionar eventos con marcas temporales correctas, preservar el contexto, distinguir evidencia de hipótesis y considerar los efectos de la rotación y los permisos de acceso. Un evento de red observado en una fecha distinta no debe utilizarse para justificar un incidente posterior, aun cuando el patrón técnico sea similar.

Finalmente, `tail -f` y `journalctl -f` muestran cómo observar un sistema en tiempo real, mientras que los filtros temporales y por unidad permiten reducir el volumen de datos durante una investigación. Estas capacidades constituyen una base directa para tareas de monitoreo, respuesta a incidentes y análisis forense en entornos Linux.

## Referencia de la práctica

Cisco Networking Academy. *Práctica de laboratorio: Registros de eventos / Ubicar archivos de registro.* Material de laboratorio CyberOps proporcionado para la actividad.